

BlockWatch

Minecraft Insider Threat Forensics Tool

IT 360 — Final Project Report

Spring 2026 | Illinois State University

Team Members

Abhi Jannu & Logan Carmody

Repository: github.com/lolo10411/IT_360_Final_Project_Spring_2026

1. Executive Summary

BlockWatch is a Python-based digital forensics tool developed as the IT 360 final project for the Spring 2026 semester at Illinois State University. The tool applies core digital forensics principles to investigate potential insider threat activity within Minecraft server environments. By treating a Minecraft server as an application system, BlockWatch performs automated log analysis, evidence integrity verification, suspicious activity detection, timeline reconstruction, and optional AI-assisted triage.

This report describes the problem BlockWatch addresses, the approach taken, the features implemented, and the technical design of the solution. The project demonstrates the application of IT security and forensics concepts to a novel, accessible domain — online gaming — making the principles practical and engaging.

2. Problem Statement

Minecraft multiplayer servers can be targets of insider threats, where trusted users with operator or administrator privileges abuse their access in harmful ways. Unlike external attacks, insider threats are harder to detect because the actions may initially appear to be legitimate administrative behavior.

Common insider threat indicators on a Minecraft server include:

- Unauthorized granting or revoking of operator privileges
- Spawning excessive items or resources (item inflation)
- Forced teleportation of players without consent
- Banning, kicking, or whitelisting players without authorization
- Changing game modes to gain unfair advantages

Manual review of Minecraft server logs is time-consuming, inconsistent, and prone to missing critical evidence — especially when suspicious events are interspersed with normal administrative actions. BlockWatch automates this process to produce consistent, repeatable forensic investigations.

3. Project Overview & Objectives

BlockWatch addresses the insider threat problem by providing an automated pipeline that ingests Minecraft server log files and produces a structured forensic report. The tool is designed with the following objectives:

- Automate the review of Minecraft server logs in a repeatable way
- Identify and flag suspicious administrative commands and log phrases
- Assign risk levels to flagged events for prioritized investigation
- Preserve evidence integrity through cryptographic hashing
- Reconstruct a timeline of suspicious activity for investigators
- Optionally leverage AI (OpenAI ChatGPT) to explain suspicious events

The project is implemented primarily in Python, and uses the OpenAI API as an optional dependency for AI-assisted analysis.

4. Repository Structure

The project repository is organized into the following directories:

Directory / File	Purpose
src/	Core Python source code for the forensics engine
data/	Sample Minecraft server log files for testing (data/sample_log.txt)
output/	Generated reports, CSV timelines, and analysis results
docs/	Project documentation and supporting materials
requirements.txt	Python dependencies (openai)
.gitignore	Excludes API keys, virtual environments, and output artifacts

README.md	Project overview, usage instructions, and feature documentation
-----------	---

5. Features & Capabilities

BlockWatch implements a comprehensive set of forensic capabilities organized into the following areas:

5.1 Log Parsing & Input Flexibility

- Parses Minecraft server log files from a user-specified path
- Can scan a Minecraft server's latest.log file directly without needing to copy it into the project folder
- Handles both slash-command style log entries and real Minecraft server log phrases

5.2 Suspicious Activity Detection

BlockWatch flags events based on two categories of indicators:

Category	Indicators Detected	Threat Relevance
Slash Commands	/give, /op, /deop, /gamemode, /tp, /kill, /ban, /pardon, /whitelist, /kick	Direct admin command abuse
Log Phrases	Gave, Set own game mode, Set game mode, Teleported, server operator, no longer a server operator, Banned, Unbanned, Kicked, Whitelist	Evidence of actions in vanilla server logs

These indicators were selected because they represent the most common vectors for privileged access abuse, privilege escalation, unauthorized item generation, forced player movement, and moderation misuse on a Minecraft server.

5.3 Risk Level Assignment

- Each flagged event is automatically assigned a risk level (e.g., Low, Medium, High)
- Risk levels prioritize investigator attention toward the most impactful events
- Enables faster triage in large log files with thousands of entries

5.4 Evidence Integrity

- SHA-256 cryptographic hashes are calculated for log evidence

- Hashes ensure that evidence has not been tampered with after collection
- Supports the chain of custody requirements in digital forensics investigations

5.5 Reporting & Timeline Reconstruction

- Generates a structured suspicious activity report in human-readable format
- Produces a CSV timeline of all flagged events, suitable for import into spreadsheet or SIEM tools
- Reconstructs a chronological incident timeline from log timestamps and actor attribution

5.6 AI-Assisted Forensic Analysis (Optional)

BlockWatch integrates optionally with the OpenAI API to provide AI-assisted analysis of suspicious log entries:

- Sends flagged Minecraft log entries to ChatGPT for forensic interpretation
- AI provides risk interpretation, contextual explanation, and recommended follow-up evidence
- Runs safely without an API key using the --no-ai flag, skipping AI analysis gracefully
- API keys are never committed to the repository (enforced via .gitignore)

6. Technical Implementation

Technical Attribute	Detail
Primary Language	Python (95.4% of codebase)
Supporting Language	PowerShell (4.6% of codebase)
External Dependency	openai (via requirements.txt)
Evidence Hashing	SHA-256 (cryptographic integrity)
AI Integration	OpenAI ChatGPT API (optional, --no-ai flag available)
Output Formats	Human-readable text report, CSV timeline
Log Compatibility	Minecraft server log format (latest.log and custom paths)
Version Control	Git / GitHub (41 commits, main branch)

The tool is designed to be run from the command line and is self-contained, requiring only Python and an optional OpenAI API key. The modular architecture separates log parsing, detection, hashing, and reporting into distinct components, making the codebase maintainable and extensible.

7. Digital Forensics Concepts Applied

BlockWatch directly applies the following core digital forensics concepts taught in IT 360:

Forensics Concept	How BlockWatch Applies It
Log Analysis	Automated parsing and scanning of Minecraft server log files for IOCs
Evidence Integrity	SHA-256 hashing of log evidence to detect tampering
Suspicious Activity Detection	Rule-based flagging of admin commands and log phrases indicating misuse
Timeline Reconstruction	Chronological ordering of flagged events with timestamp and actor extraction
Risk Triage	Risk level assignment to prioritize investigator effort
Insider Threat Investigation	Focus on privileged user actions that deviate from normal behavior
AI-Assisted Analysis	Optional LLM-based interpretation of flagged events for context and guidance
Chain of Custody	Evidence hashing and structured reporting support repeatable, defensible investigations

8. Insider Threat Application Context

While Minecraft may appear to be a non-traditional forensics domain, the insider threat patterns it exhibits are directly analogous to real-world enterprise scenarios:

Minecraft Scenario	Real-World Equivalent
Operator granting /op to unauthorized player	Privilege escalation via unauthorized admin account creation
/give used to spawn excessive items	Data exfiltration or resource theft by privileged insider
/tp used to move players without consent	Unauthorized access to restricted systems or data
/ban or /kick used to harass players	Insider using admin access to disrupt operations or target users
Game mode abuse (creative mode)	Bypassing access controls to gain unrestricted system access

By investigating a Minecraft server using real forensics techniques, the project demonstrates that the methodology and tooling of digital forensics are platform-agnostic and transferable to any system that generates logs.

9. Results

BlockWatch was tested using both a sample Minecraft log file and a real Minecraft server log generated from a local Java server. The real server log included normal activity such as player login events as well as suspicious administrative actions including item spawning, gamemode changes, teleportation, operator permission changes, banning, and unbanning.

During testing, BlockWatch successfully detected suspicious Minecraft activity from the real log file. The tool identified real server log phrases such as “Gave,” “Set own game mode,” “Teleported,” “server operator,” “Banned,” and “Unbanned.” This was important because real Minecraft logs did not always record commands exactly as slash commands. For example, a command such as /give appeared in the log as a phrase describing the result of the command.

The generated suspicious activity report included the evidence file path, report timestamp, SHA-256 hash, command summary, risk summary, incident timeline, and detailed flagged events. Each flagged event included the line number, timestamp, actor, detected command or phrase, risk level, and original log entry. The CSV timeline also exported the flagged events in a structured format that could be opened in Excel or reused in another analysis tool.

The AI-assisted analysis feature was also tested successfully using the OpenAI API. When enabled, suspicious events were sent to the AI module after being flagged by the scanner. The AI generated short forensic explanations describing why the event may matter in an insider threat investigation and what follow-up evidence should be reviewed. The AI output was included as supporting analysis, while the original Minecraft log entry remained the primary evidence.

Overall, the results showed that BlockWatch was able to complete the intended forensic workflow: identify evidence, preserve integrity through hashing, detect suspicious activity, reconstruct a timeline, classify risk, generate reports, and optionally provide AI-assisted triage.

10. Lessons Learned

One of the biggest lessons learned during development was that sample data and real log data don't always look the same. Our first version of BlockWatch primarily searched for slash commands such as /give, /op, and /gamemode, commands that were manually made in test log files. However, when the tool was tested against actual real Minecraft server logs, those same actions appeared as descriptive log phrases such as “Gave,” “Set own game mode,” “Teleported,” and “server operator.” Because of this, the detection logic had to be expanded to support both slash-command style entries and real Minecraft server log phrases.

Another lesson we learned was the importance of evidence integrity. Adding SHA-256 hashing made the tool more closely follow digital forensics principles because the analyzed log file can be verified later. Even though this project used Minecraft logs, the same evidence preservation concept applies to real forensic investigations, like the ones explored in our labs.

A third lesson learned was that timeline reconstruction makes log analysis much more useful. A simple list of flagged events is helpful, but adding timestamps, actors, risk levels, and CSV output made the findings easier to understand and present. This helped turn the tool from a basic keyword scanner into a more complete forensic triage tool.

The project also showed us that AI should support an investigation rather than replace it. The OpenAI integration was useful for explaining why suspicious events may matter, but the actual forensic evidence remained in the original log entry, timestamp, actor, SHA-256 hash, and generated report. The AI feature was designed as optional, so the core tool still works without an API key.

11. Conclusion

BlockWatch successfully delivers a functional, purpose-built digital forensics tool for investigating insider threat activity on Minecraft servers. The project covers the full forensics investigation pipeline: ingesting evidence, detecting suspicious activity, preserving integrity, reconstructing timelines, and producing investigative reports — with an optional AI layer for deeper triage.

The project demonstrates strong command of IT 360 course concepts including log analysis, evidence integrity, timeline reconstruction, risk assessment, and insider threat detection. The optional AI integration adds a forward-looking dimension by showing how large language models can assist human investigators in interpreting complex log data.

BlockWatch was built collaboratively by Abhi Jannu and Logan Carmody, with 41 commits reflecting active, iterative development throughout the Spring 2026 semester.

References

- Project Repository: https://github.com/lolo10411/IT_360_Final_Project_Spring_2026
- OpenAI API Documentation: <https://platform.openai.com/docs>
- Minecraft Server Log Format: <https://minecraft.wiki/w/Server.properties>
- NIST Guide to Integrating Forensic Techniques into Incident Response (SP 800-86)
- CERT Insider Threat Definition and Overview: <https://www.sei.cmu.edu/solutions/insider-threat>